

Thermal-Aware Fixed-Outline 3-D IC Floorplanning: An End-to-End Learning-Based Approach

Wenbo Guan^{1b}, Xiaoyan Tang^{1b}, *Member, IEEE*, Hongliang Lu^{1b}, *Senior Member, IEEE*, Yuming Zhang^{1b}, *Senior Member, IEEE*, and Yimen Zhang^{1b}, *Senior Member, IEEE*

Abstract—High temperature and temperature nonuniformity pose significant challenges in 3-D integrated circuits (3-D ICs). Numerous studies have explored thermal issues in 3-D IC floorplanning. However, most existing handcrafted heuristic algorithms suffer from long iteration cycles, resulting in inefficient thermal management and no guarantee of optimal performance. In addition, with the increasing complexity of modern integrated circuit design, current floorplanning techniques encounter the “curse of dimensionality” and struggle to optimize large-scale cases. To address these challenges, this article proposes a novel end-to-end learning-based approach for thermal-aware fixed-outline 3-D IC floorplanning. In the tier assignment stage, we utilize a deep k -means clustering algorithm to allocate modules to different tiers, aiming to achieve an improved cross-tier power distribution. In the global distribution (GD) stage, we formulate the floorplanning problem as a Markov decision process (MDP). By combining graph convolutional networks (GCNs) with a multiagent deep reinforcement learning (MADRL) algorithm, we optimize the positions of modules and through-silicon vias (TSVs), while incorporating an attention mechanism in the centralized critic to enhance cooperation among agents. Finally, in the TSV assignment (TA) stage, we refine the TSV positions using the MADRL algorithm, further reducing wirelength and temperature in 3-D ICs. Experimental results demonstrate that our proposed approach outperforms state-of-the-art heuristic-based 3-D IC floorplanner in terms of wirelength and temperature optimization.

Index Terms—3-D integrated circuits (3-D ICs), deep k -means clustering, floorplanning, graph convolutional network (GCN), multiagent deep reinforcement learning (MADRL).

I. INTRODUCTION

THREE-DIMENSIONAL integrated circuits (3-D ICs) provide a promising solution to sustain the progress projected by Moore’s law in 2-D scaling. By utilizing stacked layers, 3-D ICs effectively reduce chip area and wirelength, leading to benefits such as decreased interconnect power, increased bandwidth, and reduced routing congestion, ultimately improving overall circuit performance.

Manuscript received 21 June 2023; revised 15 August 2023 and 6 September 2023; accepted 29 September 2023. Date of publication 10 October 2023; date of current version 28 November 2023. This work was supported by the National Defense Science and Technology Foundation Strengthening Program under Grant 2019-XXXX-XX-236-00. (Corresponding author: Hongliang Lu.)

The authors are with the School of Microelectronics, Xidian University, Xi’an 710071, China (e-mail: hllv@mail.xidian.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TVLSI.2023.3321532>.

Digital Object Identifier 10.1109/TVLSI.2023.3321532

Three-dimensional IC technology offers numerous advantages, yet it also presents notable challenges, particularly in thermal management. The vertical stacking of device layers leads to increased power density, exacerbating thermal issues. Moreover, the thermal conductivity of the dielectric insulation layers between the devices is significantly lower than that of silicon and metal [1]. Therefore, the thermal constraint is a considerable obstacle that must be addressed in the design of 3-D ICs.

A. Previous Works

Extensive research has been conducted on thermal issues in 3-D IC floorplanning, leading to the proposal of diverse models and algorithms aimed at temperature optimization. Cong et al. [1] introduced a unique representation for 3-D floorplans, combining a transitive closure graph with a bucket structure and utilizing a resistive thermal model to measure temperature. Zhou et al. [2] presented a scalable 3-D floorplanning algorithm that considers temperature and leakage, incorporating force-directed techniques and power-thermal analysis to address the leakage-thermal feedback loop. Wong and Lim [3] employed an array of sequence pairs to represent module positions and utilized a random walk-based thermal analysis procedure for temperature measurement. Xiao et al. [4] proposed a two-phase simulated annealing (SA) method to handle the fixed-outline constraint in thermal-aware fixed-outline 3-D floorplanning, employing an approximate thermal model. Xu and Chen [5] used the partitioned sequence pair to represent multilayer 3-D floorplans, estimating chip temperature during floorplanning through bilinear interpolation based on block locations and their thermal profiles. Ni et al. [6] proposed a temperature-aware floorplanning approach for fixed-outline designs, assigning tiers based on module power density and strategically placing thermal modules near the heat sink. The floorplan of each tier was represented using a B*-tree, and the SA algorithm was employed for efficient floorplanning.

Due to high thermal conductivity, Knechtel et al. [7] used the corner block list [8] to represent the floorplan and applied the guided-SA algorithm for floorplanning. To handle the simultaneous placement of through-silicon vias (TSVs) and modules, they clustered the TSVs into TSV islands and estimated temperature using the power blurring method [9].

Addressing the heat concern, Chen et al. [10] proposed a temperature-aware architecture utilizing TSV islands to enhance heat conductivity across the entire 3-D chip. However, the adoption of TSV islands may introduce drawbacks such as increased delays and signal integrity issues due to TSV coupling when compared to single TSVs. Another prevalent approach in thermal-aware planning for fixed-outline 3-D ICs is the insertion of thermal TSVs (TTSVs) [3], [11]. Ren et al. [12] introduced a hierarchical method based on the normalized Polish expression to optimize the floorplan of a 3-D multicore processor following the Nehalem architecture. This method focused on arranging TTSVs, which facilitate heat transfer between tiers without carrying signals. Since the use of TTSVs increases the distance between modules and induces longer wirelength, they optimize the placement of TTSVs while lowering temperature.

Unlike the above-mentioned approaches, Lin et al. [13] proposed an advanced three-stage approach for thermal-aware fixed-outline floorplanning that handles mixed-type modules. They optimize power distribution using a hyperparameter-optimized Fiduccia Mattheyses (FM) algorithm in tier assignment. In the global distribution (GD) stage, modules and TSVs are distributed across tiers to minimize wirelength and chip temperature. The TSV assignment (TA) stage refines TSV positions tier by tier using the minimum cost maximum flow (MCMF) algorithm. This comprehensive approach effectively considers thermal aspects in floorplanning.

B. Motivation

Numerous studies have proposed thermal-aware algorithms for 3-D IC floorplanning, primarily relying on meta-heuristics. However, these approaches present two major challenges. First, the development of manually crafted heuristics based on expert knowledge is laborious and nonoptimal, requiring substantial domain expertise and trial-and-error. Second, scalability becomes an obstacle due to the increasing complexity of modern IC designs, making it difficult to represent state spaces for larger problems. Consequently, existing 3-D IC floorplanning algorithms struggle to effectively optimize large-scale cases, encountering the curse of dimensionality [14]. Therefore, the development of new methodologies for 3-D IC floorplanning is necessary. To address these challenges, reinforcement learning (RL) has emerged as a potential solution, enabling agents to learn from interactions with the environment and explore the design space effectively [15]. By training an RL agent on a single circuit, it can be directly applied to circuits of various sizes, reducing simulation costs and eliminating the need for redesigning from scratch. Although RL techniques have been successfully applied to various aspects of circuit design, including tier partitioning [16], 2-D floorplanning [17], [18], [19], logic synthesis [20], chip placement [17], [19], [21], global routing [22], and detailed routing [23], their application in 3-D IC floorplanning for wirelength and temperature optimization remains unexplored. Thus, investigating the utilization of RL in 3-D IC floorplanning represents a promising avenue to advance the field and achieve improved optimization results.

C. Our Contribution

This article presents an efficient end-to-end learning-based approach for solving thermal-aware fixed-outline 3-D IC floorplanning with mixed-type modules. In the tier assignment stage, we use a power density-aware deep k -means clustering method to group modules based on their power density. These clusters are then assigned to tiers, with higher power density modules placed closer to the heat sink. This is an unsupervised machine learning (ML) algorithm that can successfully cluster high-dimensional sample data. In the GD stage, a graph convolutional network (GCN) is employed as an encoder in a multiagent system to optimize module and TSV placements. GCN transfers knowledge across different netlist topologies using low-dimensional vectors. Treating each tier as an agent and utilizing a multiagent deep reinforcement learning (MADRL) algorithm, we decompose the floorplanning problem, reducing dimensionality. In 3-D ICs, inter-tier interactions affect wirelength and temperature, requiring agents to adapt strategies based on teammates' approaches. To address this, we introduce attention multiagent deep deterministic policy gradient (ATT-MADDPG) [24], an attention mechanism in the centralized critic, enhancing collaboration and achieving further optimization in wirelength and temperature reduction. In the TA stage, we refine the locations of the TSVs, thereby further reducing the wirelength and temperature in the 3-D IC.

Distinguishing itself from previous methods, we present an innovative end-to-end learning-based approach for thermal-aware 3-D IC floorplanning. The key technical contributions of this work are listed as follows.

- 1) We employ the deep k -means clustering method to effectively cluster modules and achieve improved cross-tier power distribution.
- 2) To address the 3-D IC floorplanning problem, we formulate it as an RL problem and utilize the MADRL algorithm. We integrate GCN as an encoding layer within a multiagent system, enabling effective netlist graph embedding. In addition, we introduce an attention mechanism in the centralized critic to model and leverage teammate policies.
- 3) We also formulate the TA problem as an RL problem and continue to use the MADRL algorithm to optimize the TSVs positions.
- 4) Experimental results demonstrate the superior performance of our proposed method compared to the state-of-the-art heuristic-based thermal-aware 3-D IC floorplanning approach.

The remainder of this article is organized as follows. Section II presents the preliminary information. Section III presents the problem formulation. The stages of our methodology are described in Sections IV–VI, respectively. Experimental results are shown in Section VII. Finally, Section VIII concludes our work.

II. PRELIMINARIES

This section provides an overview of the background of RL and the power blurring-based temperature calculation method.

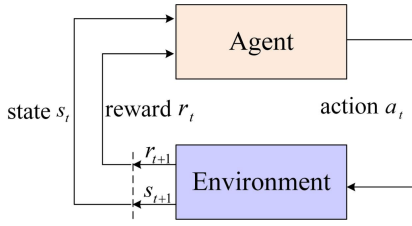


Fig. 1. Typical framework of an MDP.

A. Reinforcement Learning

RL is a learning process aimed at maximizing a numerical reward through the mapping of states to actions or action distributions [15]. This process is commonly formulated as a Markov decision process (MDP) comprising four essential components.

- 1) A set of possible environment states $\mathcal{S}$.
- 2) A set of actions $\mathcal{A}$ that can be taken by the agent.
- 3) A state transition model, $P_{ss'}^a$, representing the probability of transitioning from state $s \in \mathcal{S}$ to $s' \in \mathcal{S}$ when action $a \in \mathcal{A}$ is executed.
- 4) A reward function, R_s^a , providing numerical feedback as a reward when an action $a \in \mathcal{A}$ is performed in a state $s \in \mathcal{S}$.

The MDP framework is based on the Markov assumption, assuming that future state probabilities depend solely on the current state, rendering the past state sequence irrelevant to the environment. Fig. 1 illustrates the typical flow of an MDP, where the agent begins at state s_t executes action a_t , transitions to state s_{t+1} , and receives reward r_t . The agent's trajectory is characterized by successive episodes consisting of state-action-reward sequences.

The primary goal of an MDP is to determine an optimal policy π , which maps states from the state space to actions from the action space $\pi: \mathcal{S} \rightarrow \mathcal{A}$, with the objective of maximizing the cumulative rewards $G_t = \sum_{i=0}^{\infty} \gamma^i r_{t+i}$. The discount factor γ is used to account for the influence of delayed future rewards, and t denotes a specific time step. In practical scenarios, the Q value function $Q^\pi(s, a)$ is commonly utilized, and it is defined as follows:

$$Q^\pi(s, a) = \mathbb{E}_\pi[G|S = s, A = a] \quad (1)$$

then the optimal policy is derived by $\pi^* = \arg \max_\pi Q^\pi(s, a)$.

Policy gradient methods [25] are used to directly learn the parameterized policy $\pi_\theta = \pi(a|s; \theta)$ as an approximation for any given policy π . This is achieved by employing a deep neural network (DNN) $Q(s, a; w)$ to approximate $Q^\pi(s, a)$, allowing the utilization of actor-critic algorithms [26]. During training, both the parameterized actor $\pi = \pi(a|s; \theta)$ and the critic $Q(s, a; w)$ are utilized, while only the actor $\pi = \pi(a|s; \theta)$ is required during execution. This advantage is leveraged to train decentralized policies in our approach.

The deterministic policy gradient (DPG) [27] algorithm is an actor-critic approach in which the actor employs a deterministic policy μ_θ . Deep DPG (DDPG) [28] extends this algorithm by utilizing DNNs to approximate $\mu_\theta(s)$ and $Q(s, a; w)$. DDPG is an off-policy method that incorporates a target network and experience replays to enhance training

stability and data efficiency. The updated equations for the critic and actor can be summarized as follows:

$$\delta = r + \gamma Q(s', a'; w^-) \big|_{a'=\mu_{\theta^-}(s')} - Q(s, a; w) \quad (2)$$

$$L(w) = \mathbb{E}_{(s,a,r,s') \sim D}[(\delta)^2] \quad (3)$$

$$\nabla_\theta J(\theta) = \mathbb{E}_{s \sim D}[\nabla_\theta \mu_\theta(s) \nabla_a Q(s, a; w) \big|_{a=\mu_\theta(s)}] \quad (4)$$

where the replay buffer D stores recent experience tuples (s, a, r, s') . The target networks, $Q(s, a; w^-)$ and $\mu_{\theta^-}(s)$, utilize parameters w^- and θ^- , respectively, which are periodically updated by copying the parameters w and θ .

B. Power Blurring Method

This section introduces the power blurring method [9], a highly efficient and effective approach for temperature calculation. By utilizing a thermal mask constructed from the impulse response of a chip or layer to a point source of power, the method ensures accurate temperature estimation. The detailed procedure for constructing the thermal mask is described in [13]. To further enhance temperature estimation accuracy, the method applies the image method, an error correction step. Initially, a power distribution map is generated by dividing each tier of the 3-D IC into regular slots and estimating power consumption in each slot. Each slot within the power distribution map is treated as an individual power source. By convolving the power distribution map with the thermal mask, a temperature map is obtained.

III. PROBLEM DEFINITION

In this study, we consider a set $M = \{m_i | i \in \mathbb{Z}^+, 1 \leq i \leq n\}$ of n modules, each defined by a triplet (w_i, h_i, α_i) representing width, height, and aspect ratio. These modules can be classified into two types: hard modules with fixed aspect ratios and soft modules allowing for variable aspect ratios. The position of the module m_i in a 2-D plane is indicated by the lower-left coordinate (x_i, y_i) , while its tier in a 3-D IC is represented by z_i , satisfying $1 \leq z_i \leq K$, where K signifies the total number of tiers in the 3-D IC.

Our objective is to perform floorplanning on a 3-D IC, taking into account a predefined number of tiers and a given netlist. We aim to optimize the position and tier assignment for each module and TSV, ensuring adherence to the fixed outline. Avoiding overlaps between modules, modules and TSVs, as well as TSVs themselves, is a critical requirement. In addition, we consider both wirelength and temperature for the 3-D IC. To guide our study, we establish the following assumptions: 1) we adopt the face-to-back (F2B) bonding style; 2) I/O bumps are located below tier 1; and 3) only one TSV is inserted between two consecutive tiers in a 3-D net.

IV. POWER DENSITY-AWARE TIER ASSIGNMENT STAGE

In this section, we present a power density-aware deep k -means clustering algorithm designed to mitigate the temperature of the 3-D IC by intelligently allocating modules across multiple tiers.

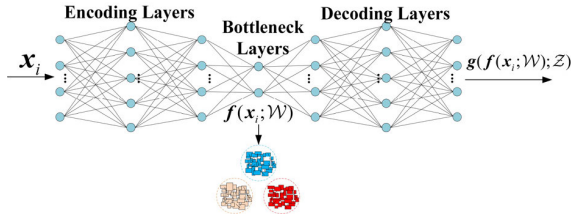


Fig. 2. Structure of the proposed deep clustering network (DCN).

A. DCN Structure and Problem Formulation

The power distribution across tiers plays a crucial role in the temperature management of 3-D ICs. To optimize the power distribution and clustering of modules, we utilize the deep k -means algorithm [29]. Fig. 2 presents the structure of our DCN. Our approach involves considering a 6-D vector, encompassing the power density of each module i , as a sample. The centroid of each tier is determined based on the target power density. Encoding layers are employed to transform the raw data into a lower-dimensional space, and the decoding layers reconstruct the data from this latent space. By minimizing the reconstruction error between the encoding and decoding layers, we train the mapping function to obtain high-quality feature representations. The k -means clustering takes place at the bottleneck layers. Our approach simultaneously optimizes the forward network, decoding network, and k -means cost. In our experiments, the decoding network mirrors the encoding network, and rectified linear units (ReLUs) [30] are used as activation functions for both networks. Referring to the network structure illustrated in Fig. 2 as the DCN, we leverage the concept of stacked autoencoders (SAEs) to successfully cluster high-dimensional sample data in the bottleneck layers using the k -means algorithm.

The DCN takes a 6-D vector $[w_i, h_i, A_i, p_i, pd_i, cn_i]$ as input, denoted as $\mathbf{x}_i$. Here, $[w_i, h_i, A_i, p_i]$ represent the width, height area, and power of module i , respectively. $pd_i = p_i/A_i$ represents the power density of module i , while cn_i represents the number of modules connected to module i . Let $R = \{r_1, \dots, r_k\}$ be the power distribution among the tiers in the 3-D IC, where r_k represents the ratio of the power of tier k to the total module power, and $\sum_{k=1}^K r_k = 1$. The power density of each tier is denoted as $d_k = r_k p_{\text{total}}/A_{\text{fixed}}$, where p_{total} and A_{fixed} represent the total power consumption and the area of the fixed-outline in 3-D IC, respectively.

Based on the previous discussion, we establish the optimization objective as follows:

$$\begin{aligned} \mathcal{P}1: \quad & \min_{\mathcal{W}, \mathcal{Z}, \mathbf{D}, \{s_i\}} \sum_{i=1}^N \left(\ell(\mathbf{g}(\mathbf{f}(\mathbf{x}_i; \mathcal{W}); \mathcal{Z}), \mathbf{x}_i) \right. \\ & \quad \left. + \frac{\lambda}{2} \|\mathbf{f}(\mathbf{x}_i; \mathcal{W}) - \mathbf{D}\mathbf{s}_i\|_2^2 \right) \\ & \text{s.t. } s_{j,i} \in \{0, 1\}, \quad \mathbf{1}^T \mathbf{s}_i = 1 \quad \forall i, j. \end{aligned} \quad (5)$$

Here, $\mathbf{s}_i$ represents the assignment vector for data point i , where it has a single nonzero element. $s_{j,i}$ corresponds to the j th element of $\mathbf{s}_i$, while $\mathbf{d}_k$ denotes the centroid of the k th cluster, represented by the k th column of $\mathbf{D}$. The mapping $\mathbf{f}(\mathbf{x}_i; \mathcal{W})$ signifies the nonlinear transformation of $\mathbf{x}_i$ through the forward network, with $\mathcal{W}$ representing the parameter set characterizing this mapping. Similarly, the

mapping $\mathbf{g}(\mathbf{f}(\mathbf{x}_i; \mathcal{W}); \mathcal{Z})$ represents the nonlinear transformation of $\mathbf{f}(\mathbf{x}_i; \mathcal{W})$ through the backward network, with $\mathcal{Z}$ denoting the associated parameter set. The loss function $\ell(\cdot)$ measures the reconstruction error. To ensure a meaningful alignment between $\mathbf{g}(\mathbf{f}(\mathbf{x}_i; \mathcal{W}); \mathcal{Z})$ and $\mathbf{x}_i$ under specific metrics, we employ the least-squares loss $\ell(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2^2$ as a preventive measure against trivial solutions. Introducing the regularization parameter $\lambda \geq 0$ allows us to strike a balance between the reconstruction error and the exploration of k -means-friendly latent representations.

B. Optimization Procedure

1) *Initialize Network Parameters:* To tackle the challenging nonconvex optimization problem mentioned earlier, proper initialization plays a crucial role. We address this by adopting a layer-wise pretraining approach to train the autoencoders and initialize the network parameters $(\mathcal{W}, \mathcal{Z})$ [31]. Initially, a single-layer autoencoder trains the first layer of the encoding network and the last layer of the decoding network. The output of the trained forward layer then serves as the input for training subsequent pairs of encoding and decoding layers, following a similar process. This layer-wise pretraining procedure continues iteratively until completion. After achieving successful pretraining, we utilize k -means clustering on the bottleneck layer outputs to determine the initial values of $\mathbf{D}$ and $\{s_i\}$.

2) *Alternating Stochastic Optimization:* The discrete set constraint of the block variable $\{s_i\}$ prevents direct application of the widely used stochastic gradient descent (SGD) algorithm for joint optimization of $\mathcal{W}$, $\mathcal{Z}$, $\mathbf{D}$, and $\{s_i\}$. To overcome this challenge, we employ a combination of alternating optimization and SGD, enabling the optimization of one parameter set while keeping the other two fixed. Initially, we focus on updating the network parameters $(\mathcal{W}, \mathcal{Z})$ given $\mathbf{D}$ and $\{s_i\}$. To facilitate SGD updates, we transform (5) into a sub-problem expressed as follows:

$$\min_{\mathcal{W}, \mathcal{Z}} L^i = \ell(\mathbf{g}(\mathbf{f}(\mathbf{x}_i)), \mathbf{x}_i) + \frac{\lambda}{2} \|\mathbf{f}(\mathbf{x}_i) - \mathbf{D}\mathbf{s}_i\|_2^2. \quad (6)$$

For brevity, we simplify the notation $\mathbf{f}(\mathbf{x}_i; \mathcal{W})$ and $\mathbf{g}(\mathbf{f}(\mathbf{x}_i; \mathcal{W}); \mathcal{Z})$ to $\mathbf{f}(\mathbf{x}_i)$ and $\mathbf{g}(\mathbf{f}(\mathbf{x}_i))$, respectively. The gradient of the aforementioned function with respect to the network parameters is computed as follows:

$$\nabla_{\mathcal{X}} L^i = \frac{\partial \ell(\mathbf{g}(\mathbf{f}(\mathbf{x}_i)), \mathbf{x}_i)}{\partial \mathcal{X}} + \lambda \frac{\partial \mathbf{f}(\mathbf{x}_i)}{\partial \mathcal{X}} (\mathbf{f}(\mathbf{x}_i) - \mathbf{D}\mathbf{s}_i) \quad (7)$$

where $\mathcal{X} = (\mathcal{W}, \mathcal{Z})$ represents the set of network parameters, and the gradients $(\partial \ell / \partial \mathcal{X})$ and $(\partial \mathbf{f}(\mathbf{x}_i) / \partial \mathcal{X})$ can be efficiently obtained through back propagation (BP) [32]. Subsequently, the update of network parameters can be expressed as follows:

$$\mathcal{X} \leftarrow \mathcal{X} - \kappa \nabla_{\mathcal{X}} L^i \quad (8)$$

where κ is a predefined learning rate.

When network parameters $\mathcal{X} = (\mathcal{W}, \mathcal{Z})$ and $\mathbf{D}$ are fixed, we can update the indicator vector $\mathbf{s}_i$ of the current sample online

$$s_{j,i} \leftarrow \begin{cases} 1, & \text{if } j = \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{f}(\mathbf{x}_i) - \mathbf{d}_k\|_2 \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

Algorithm 1 Deep k -Means Clustering Algorithm

Input: An circuit netlist, k : number of clusters, number of hidden layers and neurons

Output: $\{C_1, \dots, C_k\}$: k clusters.

1: Initialize network parameter $X = (W, Z)$ based on pre-training results D and $\{s_i\}$

2: **repeat**

3: $\{C_1, \dots, C_k\} = \arg \min_{\Omega, Z, D, \{s_i\}} \sum_{i=1}^N \left(\lambda(g(f(x_i)), x_i) + \frac{\lambda}{2} \|f(x_i) - Ds_i\|_2^2 \right)$

4: update network parameters by (8)

5: update assignment by (9)

6: update centroids by (10)

7: **until** $\{C_1, \dots, C_k\}$ no longer change

Likewise, when $\mathcal{X} = (\mathcal{W}, \mathcal{Z})$ and s_i are fixed, we utilize the method proposed in [33] to update d_k . To facilitate this process, we introduce c_k^i , representing the number of times a sample has been assigned to cluster k before processing the incoming sample x_i . d_k is then updated using a simple gradient step as follows:

$$d_k \leftarrow d_k - (1/c_k^i)(d_k - f(x_i))s_{k,i} \quad (10)$$

where the gradient step size $1/c_k^i$ controls the learning rate.

To update (5), we employ an iterative minimization technique, outlined in Algorithm 1. Initially, the network parameters $\mathcal{X} = (\mathcal{W}, \mathcal{Z})$ are initialized using the pretraining outcomes of D and $\{s_i\}$ (line 1). Subsequently, modules are assigned to centroids based on the smallest loss function, derived from pretraining, leading to the determination of clusters $\{C_1, \dots, C_k\}$ (line 3). The parameters in the minimum loss function are then updated using the alternating SGD algorithm (lines 4–6). The clustering process is complete when the assignments no longer change.

V. 3-D THERMAL-AWARE GLOBAL DISTRIBUTION STAGE

This section illustrates a thermal-aware GD approach. After modules and TSVs have been assigned to different tiers, we simultaneously distribute modules and TSVs to placement regions in all tiers while optimizing the total wirelength and temperature. We begin by discussing the floorplanning problem and subsequently formulate it as an MDP. The following section outlines the proposed learning-based network architecture, while also defining network inputs that guide the decision-making process of the multiagent. To generate feature embeddings of the netlist, we introduce a GCN. Finally, we integrate the MADRL algorithm with an attention mechanism to learn optimal policies.

A. Floorplanning Definition

In this section, our objective is to optimize the positions of modules and TSVs in each tier while minimizing wirelength and temperature. Based on wirelength function $W(\cdot)$ and temperature function $T(\cdot)$, the optimization objective is defined as follows:

$$\min_{f_p} W(f_p) + \beta T(f_p) \quad (11)$$

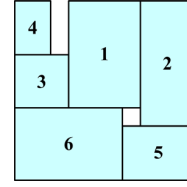


Fig. 3. Floorplan implied by SP_i ($\langle 4 \ 3 \ 1 \ 6 \ 5 \ 2 \rangle$, $\langle 6 \ 3 \ 5 \ 4 \ 1 \ 2 \rangle$), where $1 \leq i \leq K$. The sizes for the six modules are: 1(4×6), 2(3×7), 3(3×3), 4(2×3), 5(4×3), and 6(6×4), respectively.

where f_p represents a feasible floorplan solution and β is a coefficient. To calculate the temperature, we employ the power blurring method (see Section II-B), while the widely used half-perimeter wirelength (HPWL) model is utilized for wirelength approximation [34]. Recognizing the disparity in magnitudes between the wirelength and temperature objectives, we apply a normalization technique to both objectives to address this issue.

In this study, we utilize the widely researched multilayer sequence pair $(SP_1, SP_2, \dots, SP_K)$ as the floorplan representation in 3-D IC [35].

Definition 1 (Sequence Pair): An SP_i (Γ_1, Γ_2) consists of two sequences of n elements, representing a list of n modules in tier i . It establishes the geometric relations between nonoverlapping modules as follows:

$$\begin{aligned} &(\langle \dots, b_i, \dots, b_j, \dots \rangle, \langle \dots, b_i, \dots, b_j, \dots \rangle) \\ &\Rightarrow b_i \text{ is left to } b_j \\ &(\langle \dots, b_j, \dots, b_i, \dots \rangle, \langle \dots, b_i, \dots, b_j, \dots \rangle) \\ &\Rightarrow b_i \text{ is below to } b_j. \end{aligned}$$

The construction of horizontal- and vertical-constraint graphs is applied to the given SP_i [36]. By calculating the longest path length, the resulting floorplan is obtained. Notably, when there are n modules, the number of possible floorplan configurations is $(n!)^2$. The original paper proposing SP_i introduces an $\mathcal{O}(n^2)$ algorithm for transforming SP_i into a floorplan. To speed up this process, Tang et al. [37] proposed an $\mathcal{O}(n \log \log n)$ evaluation algorithm that computes weighted longest common subsequences. For example, Fig. 3 shows the corresponding floorplan for the SP_i ($\langle 4 \ 3 \ 1 \ 6 \ 5 \ 2 \rangle$, $\langle 6 \ 3 \ 5 \ 4 \ 1 \ 2 \rangle$).

B. Problem Formulation

In this stage, we address the floorplanning problem, which aims to optimize the positions of blocks and TSVs to minimize wirelength and temperature. Traditionally, local search heuristics have been employed for generating these assignments. However, we propose a novel approach by exploring the possibility of learning these heuristics through accumulated search experiences. Our main objective is to develop a local search algorithm from scratch, free from too much prior human knowledge, for exploring floorplans. This approach aims to attain high-quality solutions by effectively leveraging efficient local search heuristics.

To explore improved floorplan strategies through local search, we consider each tier as an agent and define a corresponding MDP for each agent.

- 1) *State Space*: In the floorplanning problem, the state S represents a complete solution comprising an entire SP_{*i*} and the orientation of each module.
- 2) *Action Space*: We define six types of perturbations for each agent, enabling the generation of neighboring floorplan solutions.
 - a) Swap a random pair of modules in Γ_1 .
 - b) Swap a random pair of modules in Γ_2 .
 - c) Swap a random pair of modules in both Γ_1 and Γ_2 .
 - d) Shift a randomly selected module to a new position in both Γ_1 and Γ_2 .
 - e) Rotate a randomly selected module by 90°.
 - f) Shift a randomly selected TSV to a new position within the range of $1 \leq r \leq 3$, excluding the last tier. Fig. 4 illustrates the search space for TSV positions in the GD stage.

Following the sampling of a neighboring floorplan, the RL agent determines whether to accept or reject the move, resulting in an action space size of 2.

- 3) *State Transition*: Given a floorplan solution (state), applying any of the mentioned perturbation methods causes the agent to transition to a different state. This simplifies the probabilistic aspect of our MDP formulation.
- 4) *Reward*: The goal of the floorplanning problem is to minimize both wirelength and temperature. As the agents strive to maximize their total reward, the action of maximizing cumulative rewards should be consistent with the goal of the floorplanning problem.

In this stage, we employ a reward system based on the reduction in objective cost to drive the local search. Rewards are assigned when a superior solution is found, while no reward is given for inferior solutions. To ensure consistency, the rewards are normalized to a range of [0, 1] by dividing the cost of the initial solution

$$R = \begin{cases} 0, & \text{if } f(s') \geq f(s) \\ \frac{f(s) - f(s')}{\text{baseline}}, & \text{if } f(s') < f(s) \end{cases} \quad (12)$$

where $f(\cdot)$ is the objective function shown in (11). s denotes the current solution (state), while s' denotes the neighboring solution of s . By maximizing the total reward, we effectively minimize the objective cost, as indicated in (12). Based on these principles, we formulate the floorplanning local search problem.

Problem 1: Given the MDP model described above, the RL agents aim to find their optimal policies $\pi^* = \{\pi_1^*, \dots, \pi_K^*\}$ to minimize the wirelength and temperature of the floorplan.

C. Algorithm Framework in GD Stage

RL agents aim to learn an optimal policy that maps the state space to the action space, but storing the best action for each state is infeasible due to the large number of possible states, especially in floorplanning scenarios with n modules. To address this, we propose utilizing a neural network to approximate the optimal policy. However, training such a network is challenging due to the vast state space and diverse properties of netlists. To handle these variations, we integrate a

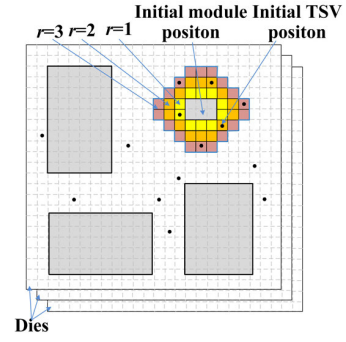


Fig. 4. Search space for TSV positions in the GD stage. The whitespace between modules within each tier is divided into $E \times F$ regular slots. Black dots indicate the initial TSV positions, while gray areas represent the modules. The yellow, orange, and pink slots indicate the available locations for perturbing TSVs corresponding to $r = 1$, $r = 2$, and $r = 3$, respectively.

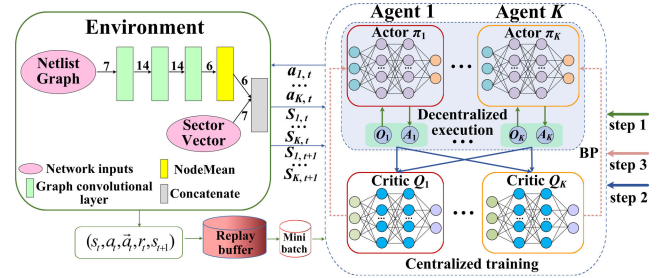


Fig. 5. Schematic of the floorplan framework in the GD stage. A GCN encodes information about the netlist graph. In the multiagent system, the policy networks and value networks of each agent output probability distributions for actions in each tier and the expected returns for the current tier's floorplan.

graph neural network (GNN) as an encoder in our architecture to encode information associated with unseen netlists. In 3-D IC floorplanning, the interactions between chip tiers and the vertical stacking of TSVs affect wirelength and temperature. Thus, each agent (representing a tier) needs to adapt its strategies based on its teammates. To tackle this challenge, we employ the ATT-MADDPG algorithm, which incorporates an attention mechanism to model and exploit teammate strategies (see Section V-F). Our proposed thermal-aware floorplan framework, as depicted in Fig. 5, efficiently optimizes the positions of modules and TSVs.

First, we propose a novel GNN for extracting information from netlists. The GNN consists of three graph convolutional layers with dimensions of 14, 14, and 6, respectively (Section V-E provides detailed descriptions of the graph convolution operations). To obtain the overall netlist graph embedding, we employ a reduced mean function in the final layer. By integrating this GNN as an encoder into our network architecture, we effectively encode information from unseen netlists into meaningful signals during inference. In addition, we incorporate other state features, such as cost-related statistical information and search step details outlined in Section V-D, into the 1-D vector by concatenating them. This vector is then passed as input to the multiagent system, with a ReLU activation function applied after each layer except the last.

Next, after embedding the graph node information, we consider each chip tier as an independent agent and employ a MADRL algorithm to optimize the positions of modules and TSVs for minimizing wirelength and temperature in 3-D ICs (see Section V-F). The critic network of each agent incorporates observations and actions from all agents to guide

updates in their respective actor networks. However, each agent's actor-network only relies on its own observations for decision-making. To enhance cooperative behavior among agents, we introduce an attention mechanism into the centralized critic (see Section V-G). This attention mechanism dynamically captures the joint policy of teammates and adapts as policies evolve, facilitating quick policy adjustments for efficient cooperation. Ultimately, the RL agents aim to discover their optimal policies $\pi^* = \{\pi_1^*, \dots, \pi_k^*\}$ to minimize wirelength and temperature in the floorplan.

D. Input Features

To optimize floorplan solutions, the agent's decision-making process involves selecting actions based on environmental information obtained after each perturbation. In this stage, several decision-making features are introduced to facilitate this process. These features include:

- 1) The objective cost of the current state;
- 2) The lowest and average objective costs observed so far;
- 3) The objective cost of a sampled neighboring state;
- 4) The area of the perturbed modules and the number of other affected modules;
- 5) A scalar value indicating the current step; and
- 6) The netlist graph, which consists of the graph adjacency matrix and node features.

By considering these decision-making features, the agent can generate action distributions and estimate the expected reward for the current floorplan solution, thereby helping to optimize the solution.

First, the primary objective of the floorplan search is cost minimization. The agent evaluates whether to accept a sampled neighbor based on the defined cost function (11), utilizing cost-related statistical information from the first three state features to enhance decision-making.

Second, each perturbation method leads to a transition to a new state. To quantify the impact of this transition, we incorporate the area of perturbed modules and the number of affected modules. An affected module indicates that its coordinates are influenced by the perturbed module.

Third, search progress is included as a feature. Despite the agent having an early stop mechanism, it remains unaware of the progress since it is considered irrelevant to local search.

Finally, the floorplanning task involves a netlist graph as input. Fig. 6(a) illustrates an input netlist with seven nets, $\eta_1 = \{A, D, F\}$, $\eta_2 = \{B, D, E\}$, $\eta_3 = \{C, E\}$, $\eta_4 = \{D, F, G\}$, $\eta_5 = \{E, G, J\}$, $\eta_6 = \{F, H, I\}$, $\eta_7 = \{G, I, J\}$. Each net forms a k -clique, where k denotes the net's size, and the edges within the clique are weighted by $2/k$ [38]. Fig. 6(b) depicts the corresponding undirected clique-based graph model $\mathcal{G}(\mathcal{V}, \mathcal{E})$. To process the netlist input, we employ a GCN to generate feature embeddings (see Section V-E). This enables the agent to learn patterns that generalize to unseen netlists.

Notably, all features are normalized to facilitate training. To achieve this, the initial state's objective cost is denoted as e_0 , and all cost-related entries are normalized using the formula $\min(1, f/e_0 - 1)$. Furthermore, these aforementioned features are concatenated into a single vector representation, which is subsequently passed to the downstream network.

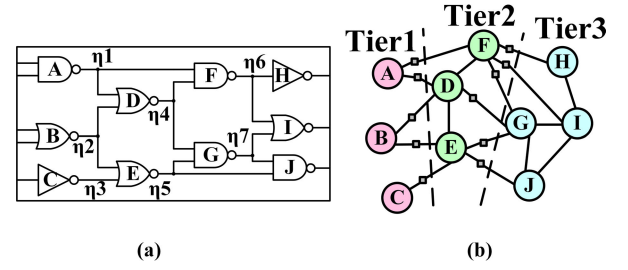


Fig. 6. (a) Input netlist. (b) Graph representation. Each pink, green, and blue node $v_m \in \mathcal{V}$ represents a circuit module on a different tier of the chip, while each gray node $v_T \in \mathcal{V}$ represents a TSV. Each edge $e \in \mathcal{E}$ represents a connection between two nodes.

E. Graph Feature Embedding

Graph-based learning is a promising and versatile approach in ML with diverse applications [39]. It involves obtaining node or graph representations, known as embedding, which are essential for performing specific tasks and can be effectively utilized by downstream models. For the floorplanning problem, a netlist can be represented as an undirected graph, as shown in Fig. 6. To extract features from the netlist graph, we employ GCN. The GCN is integrated into our proposed end-to-end network architecture as an encoder, as depicted in Fig. 5. This integration enables the network to encode pertinent information from previously unseen netlists into meaningful signals during the inference phase.

Each graph node is initially characterized by a compact 7-D vector $[A, w, h, pd, z, x, y]$. Here, $[A, w, h, pd, z, x, y]$ corresponds to the area, width, height, power density, number of tiers, and x - and y -coordinates of the associated module or TSV, respectively. Notably, the TSVs possess a width and height of $3 \mu\text{m}$, along with a power density of 0. To enable further processing, the node embeddings undergo three consecutive graph convolution layers.

In this stage, we employ a GCN to generate node embeddings. The process consists of two key steps: aggregation and encoding, expressed as follows:

$$\mathbf{h}_{\mathcal{N}(v)}^l \leftarrow \text{AGGREGATE}_l(\{\mathbf{h}_u^{l-1} \mid \forall u \in \mathcal{N}(v)\}) \quad (13a)$$

$$\mathbf{h}_v^l \leftarrow \sigma(\mathbf{W}^l \cdot \text{CONCAT}(\mathbf{h}_v^{l-1}, \mathbf{h}_{\mathcal{N}(v)}^l)) \quad (13b)$$

where AGGREGATE_l in (13a) denotes the aggregator function in layer l , applied to the neighboring nodes of each node v . After aggregating the neighboring feature vector, we first concatenate the current representation of node v , $\mathbf{h}_v^{l-1}$, with the aggregated neighborhood vector $\mathbf{h}_{\mathcal{N}(v)}^l$. Next, the concatenated vector undergoes an encoding operation involving an embedding projection and a nonlinear activation function $\sigma(\cdot)$, as shown in (13b). The weight matrix $\mathbf{W}^l$ represents the weights of layer l . The results are then passed on to the subsequent graph convolutional layer.

An example illustrating a 2-layer GCN for generating the embedding for node 1 is depicted in Fig. 7, with encoding dimensions in layer l_1 and l_2 are d_1 and d_2 , respectively. In the initial graph convolutional layer l_1 , each node's hidden representation is obtained by aggregating its own features along with those of its neighboring nodes. For example, in node 1, the aggregator function combines the features of node 1 itself and its neighboring nodes 2–4. After aggregating the features, a nonlinear transformation is applied to each

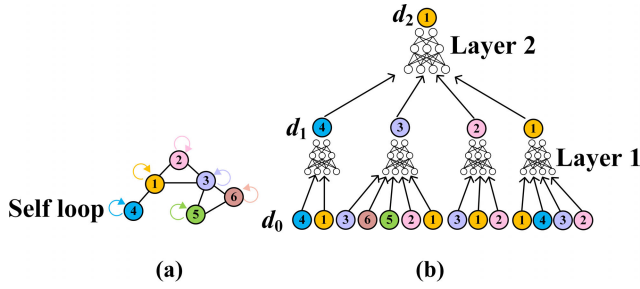


Fig. 7. Example of computing the embedding for a node with two layers. (a) Netlist graph. (b) Process of embedding computation for node 1.

node, resulting in a new feature vector of dimension d_1 . These same aggregation and encoding operations are then performed in layer l_2 . As a result, the final hidden representation of node 1 incorporates information from a broader neighborhood.

When the mean function is chosen as the aggregator function, the aggregation and encoding processes in layer l can be expressed as follows:

$$H^{l+1} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^l W^l). \quad (14)$$

Here, $\tilde{A} = A + I_N$ represents the adjacency matrix of the undirected graph $\mathcal{G}$, which includes self-connections. The identity matrix I_N and diagonal matrix $\tilde{D}$ are used, with each $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$. H^l represents the node embeddings for the entire graph. Ultimately, the embedding for the entire graph is computed as the mean of all node embeddings at the final graph convolutional layer.

F. ATT-MADDPG for Floorplanning

After embedding graph node information, we adopt a MADRL algorithm to optimize modules and TSVs placement in 3-D ICs, aiming to reduce wirelength and temperature. However, in 3-D ICs, nets often span multiple chip tiers, and the wirelength and temperature of the chip are influenced by the mutual interactions between tiers resulting from the vertical stacking of TSVs. Consequently, agents representing individual chip tiers need to adapt their strategies based on their teammates' actions, fostering effective cooperation to minimize chip temperature and wirelength.

To capture and utilize teammates' policies in cooperative multiagent systems, we employ the ATT-MADDPG algorithm proposed by [24], as illustrated in Fig. 5. ATT-MADDPG is specifically designed to adaptively model the dynamic joint policy of teammates, focusing on training decentralized policies for large-scale distributed tasks with continuous action spaces. Access to teammates' observations and actions is crucial for effective agent modeling, and ATT-MADDPG addresses this by utilizing a centralized critic to gather the required information. To efficiently process this information and model teammates' policies, an attention mechanism is integrated into the centralized critic (see Section V-G). This attention mechanism introduces a specialized structure that dynamically and adaptively captures the joint policy of teammates. As teammates adjust their policies, the associated attention weights adapt accordingly, enabling the agent to quickly adapt its own policy. This leads to efficient cooperation and collaboration among all agents.

Algorithm 2 ATT-MADDPG for GD Stage

Input: Randomly initialize critic network $Q(\langle o, \bar{o} \rangle, a, \bar{a}; w)$ and actor $\mu_\theta(o)$ with weights w and θ .

Output: An optimal policy $\pi^* = \{\pi_1^*, \pi_2^*, \dots, \pi_K^*\}$.

- 1: Initialize target network $Q(\langle o', \bar{o}' \rangle, a', \bar{a}'; w^-)$ and $\mu_{\theta^-}(o')$ with weights $w^- \leftarrow w, \theta^- \leftarrow \theta$
- 2: Initialize replay buffer D
- 3: **for** episode = 1 to M **do**
- 4: Initialize a random process $\mathcal{N}$ for action exploration
- 5: Receive initial observation state s_1
- 6: **for** $t=1$ to T **do**
- 7: for each agent i , select action $a_{i,t} = \mu_{\theta_i}(o_{i,t}) + \mathcal{N}_t$ according
- 8: to the current policy and exploration noise
- 8: Execute actions $a_t = (a_{1,t}, \dots, a_{K,t})$ and observe reward
- 8: $r_t = (r_{1,t}, \dots, r_{K,t})$ and new state s_{t+1}
- 9: Store transition $(s_t, a_t, \bar{a}_t, r_t, s_{t+1})$ in replay buffer D
- 10: $s_t \leftarrow s_{t+1}$
- 11: **for** agent $i=1$ to K **do**
- 12: Sample a random minibatch of S samples
- 13: $(s_i, a_i, \bar{a}_i, r_i, s_{i+1})$ from D
- 13: Set equation (17)
- 14: Update critic by minimizing the loss by equation (18)
- 15: Update the actor policy using the sampled policy gradient by equation (19)
- 16: **end for**
- 17: Update target networks parameters for each agent i :
- 18: $w^- \leftarrow \tau w + (1-\tau)w^-$
- 19: $\theta^- \leftarrow \tau \theta + (1-\tau)\theta^-$
- 20: **end for**
- 21: **end for**

Our approach incorporates several techniques, including the attention mechanism, replay buffer, experience replay mechanism, and neural network update method, to achieve our objectives. The replay buffer efficiently stores the state, action, and reward information of all agents at each time step. As shown in Fig. 5, we treat the tiers as individual agents in the system. The centralized critic Q_i , representing the Q value function for agent i , has access to observations and actions from all agents, while the independent actor π_i can only access its own observation o_i . Our method follows the following procedure during the training process.

Step 1: The actors π_i use their individual observations o_i to generate actions a_i and interact with the environment.

Step 2: The centralized critics estimate the Q -values Q_i by considering the observations and actions of all agents. To effectively model teammates' policies using the gathered information, we enhance the centralized critic with an attention mechanism (see Section V-G).

Step 3: After receiving reward feedback from the environment, the actors and critics undergo joint training using BP, as described in (17)–(19).

Algorithm 2 gives a high-level description of the ATT-MADDPG algorithm to solve the floorplan problem. In the initialization stage, the target networks $Q(\langle o', \bar{o}' \rangle, a', \bar{a}'; w^-)$ and $\mu_{\theta^-}(o')$ are initialized with weights w^- and θ^- , respectively, along with the initialization of the replay buffer D

(lines 1–2). The training process comprises M episodes, each consisting of T training steps. Subsequently, at the start of each episode, the noise process $\mathcal{N}$ and the initial state s_1 are randomly generated (lines 4–5). At each time step t , each agent i selects an action based on its current policy and exploration noise, leading to a reward and a transition to a new state. The transition $(s_t, a_t, \vec{a}_t, r_t, s_{t+1})$ is stored in the replay buffer D (lines 7–10). Then, S samples are randomly sampled by each agent i from the replay buffer D (line 12). The critic of each agent updates by minimizing the mean square error between the real Q value and the target Q value, while the actor adjusts its policy parameters θ based on the real Q value received from the critic. In addition, the critic employs the same error to update the state value weights w (lines 13–15). Finally, a soft update is employed to update the parameters w^- and θ^- of the target networks (lines 17–19).

G. Attention Mechanism

In order to facilitate effective collaboration and optimize the reduction of chip temperature and wirelength, it is crucial for the agent to comprehend and adapt to teammates' policies. To address this challenge, we propose the integration of an attention mechanism into the centralized critic. This enhancement enables the adaptive modeling of dynamic joint policies among teammates, ensuring that the agent can accurately capture the cooperative behaviors essential for achieving better results. By incorporating this attention mechanism, our approach enhances the agent's ability to collaborate effectively and improve overall performance in the task.

From the perspective of agent i , the outcome of action a_i in state s is influenced by the joint actions of teammates $\vec{a}_{-i}$. Our objective is to determine the optimal policy $\pi_i^* = \arg \max_{\pi_i} Q_i^{\pi_i | \vec{\pi}_{-i}}(s, a_i)$ by approximating the Q value function, $Q_i^{\pi_i | \vec{\pi}_{-i}}(s, a_i)$, associated with the joint policy of teammates. To achieve this approximation, we propose an attention mechanism, as depicted in Fig. 8, which consists of an attention module and a K -head module, with $K = |\vec{A}_{-i}|$. The K -head module generates K action conditional Q -values, denoted as $Q_i^k(s, a_i | \vec{a}_{-i}; w_i)$, where w_i is the parameters of the critic network of agent i . Specifically, $Q_i^k(s, a_i | \vec{a}_{-i}; w_i)$ is generated based on a_i and all observations $\langle o_i, \vec{o}_{-i} \rangle = \vec{o} \triangleq s$ with $\vec{a}_{-i}$ obtained from an additional hidden vector $h_i(w_i)$. The K -head module facilitates grouping of similar $\vec{a}_{-i}$ actions. Within the attention module, the attention weight vector $W_i(w_i)$ is generated by comparing $h_i(w_i)$ with all action conditional Q -values, $Q_i^k(s, a_i | \vec{a}_{-i}; w_i)$:

$$W_i^k(w_i) = \frac{\exp(h_i(w_i) Q_i^k(s, a_i | \vec{a}_{-i}; w_i))}{\sum_{k=1}^K \exp(h_i(w_i) Q_i^k(s, a_i | \vec{a}_{-i}; w_i))}. \quad (15)$$

Finally, we approximate $Q_i^{\pi_i | \vec{\pi}_{-i}}(s, a_i)$ using the following formula [24]:

$$Q_i^c(s, a_i, \vec{a}_{-i}; w_i) = \sum_{k=1}^K W_i^k(w_i) Q_i^k(s, a_i | \vec{a}_{-i}; w_i). \quad (16)$$

Once we obtain the contextual Q value $Q_i^c(s, a_i, \vec{a}_{-i}; w_i)$, we convert the multidimensional Q_i^c into a scalar real Q value

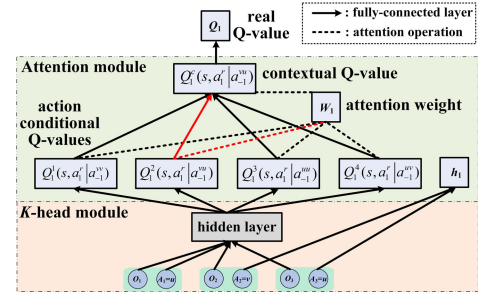


Fig. 8. Schematic of the attention mechanism. We illustrate the detailed generation of Q_1 using a three-agent example with a discrete action space $\{u, v\}$. Each agent has a preference for taking a specific action: u, v , and u , respectively. Notably, in this scenario, the second action conditional Q value Q_1^2 holds greater significance in computing the contextual Q value Q_1^c , as depicted by the thicker red links. It is important to distinguish between Q_i (real Q value), Q_i^c (contextual Q value), and Q_i^k (action conditional Q value). While Q_i^c and Q_i^k are multidimensional vectors, Q_i represents the real scalar Q value employed in (17)–(19).

Q_i , using a fully connected layer with a single output neuron, as depicted at the top of Fig. 8. The ReLU activation function is applied to the neural network. The critic network incorporates observations and actions from all agents, resulting in the real Q value Q_i , expressed as $Q_i(\langle o_i, \vec{o}_{-i} \rangle, a_i, \vec{a}_{-i}; w_i)$. Thus, we extend (2)–(4) into multiagent formulations as follows:

$$\delta_i = r_i + \gamma Q_i(\langle o'_i, \vec{o}'_{-i} \rangle, a'_i, \vec{a}'_{-i}; w_i^-) \Big|_{a'_j = \mu_{\theta_j}(o'_j)} - Q_i(\langle o_i, \vec{o}_{-i} \rangle, a_i, \vec{a}_{-i}; w_i) \quad (17)$$

$$L(w_i) = \mathbb{E}_{(o_i, \vec{o}_{-i}, a_i, \vec{a}_{-i}, r_i, o'_i, \vec{o}'_{-i}) \sim D} [(\delta_i)^2] \quad (18)$$

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{(o_i, \vec{o}_{-i}) \sim D} [\nabla_{\theta_i} \mu_{\theta_i}(o_i) \nabla_{a_i} Q_i(\langle o_i, \vec{o}_{-i} \rangle, a_i, \vec{a}_{-i}; w_i) \times |a_j = \mu_{\theta_j}(o_j)|] \quad (19)$$

where, a'_i represents the local action of agent i in the next state after s , while $\vec{a}'_{-i}$ represents the joint action of teammates of agent i in the next state after s . Similarly, o'_i and $\vec{o}'_{-i}$ are denoted. $Q_i(\langle o'_i, \vec{o}'_{-i} \rangle, a'_i, \vec{a}'_{-i}; w_i^-)$ denotes the target Q value function of the target critic network. Then, the target network parameters for the actor and critic networks are updated periodically using a soft update algorithm.

In addition, the deterministic nature of the actions generated by the policy lacks exploration of the environment. To address this limitation, we introduce an exploration policy, denoted as μ' . By incorporating noise sampled from a noise process $\mathcal{N}$ into our actor policy, we encourage exploration of the environment. This is represented by the following equation:

$$\mu'(o_{i,t}) = \mu_{\theta_{i,t}}(o_{i,t}) + \mathcal{N} \quad (20)$$

where $\mu_{\theta_{i,t}}(o_{i,t})$ denotes the deterministic policy of agent i at time step t . The selection of $\mathcal{N}$ can be tailored to fit the specific environment. In our study, we utilize a Gaussian noise process to facilitate exploration [40].

VI. THERMAL-AWARE TSV ASSIGNMENT STAGE

In order to further minimize wirelength and temperature, this section focuses on refining the locations of TSVs. We begin by providing a description of the TA problem and subsequently formulate it as an MDP. To solve this problem, we employ the ATT-MADDPG algorithm once again. It is

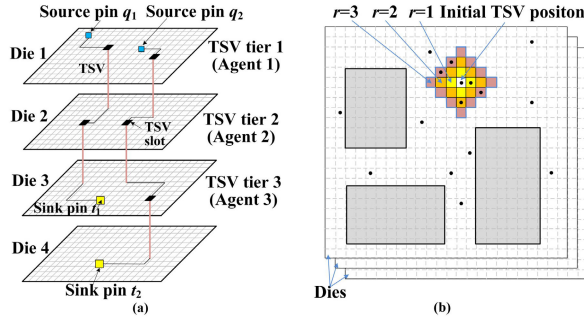


Fig. 9. (a) Two-net slot structure in four-chip die. (b) The search space for TSV positions in the TA stage. Black dots indicate the initial TSV positions, while gray areas represent the modules. The yellow, orange, and pink slots indicate the available locations for perturbing TSVs corresponding to $r = 1$, $r = 2$, and $r = 3$, respectively.

important to note that, unlike the GD stage, the agents in TA stage are represented by the TSV tiers.

A. TSV Assignment Problem Definition

The task of TA in the 3-D IC involves determining suitable locations for TSVs to establish connections between chip dies. The input for TA can be summarized as follows.

- 1) A 3-D IC comprises K chip tiers denoted as $\mathcal{B} = \{b_1, b_2, \dots, b_K\}$, with $K - 1$ corresponding TSV tiers denoted as $\mathcal{C} = \{c_1, c_2, \dots, c_{K-1}\}$. Each TSV tier represents a chip tier containing TSVs.
- 2) The whitespace between modules within each tier is divided into $E \times F$ regular slots. Each slot, referred to as g , represents a potential position for TSV placement.
- 3) The 3-D netlist contains m nets that define interconnections between modules. Each net is a two-pin configuration, $\eta_i = q_i, t_i$, where q represents the source pin transferred from the top tier using a minimum spanning tree (MST), and t denotes the sink pin [41].
- 4) Each slot g is associated with a position relative to the left bottom corner of the chip, denoted as $p_g = (x_g, y_g, z_g)$, where z_g indicates the chip die number of g . Similarly, each pin is assigned a position $p_q = (x_q, y_q, z_q)$ or $p_t = (x_t, y_t, z_t)$. Here, z_q and z_t indicate the chip die numbers of the source and sink pins, respectively, with the constraint $1 \leq z_q < z_t \leq K$.

We define the cost function for net η_i consisting of wirelength and temperature

$$\psi_i = \varsigma \times W_i + (1 - \varsigma) \times T_i \quad (21)$$

where ς denotes a user-specified parameter. W_i and T_i are the costs of wirelength and temperature, respectively. The temperature cost function is outlined as follows:

$$T_i = \sum_{k=1}^{K-1} (T_{\max}^k - T_{g(u,v)}^k) \quad (22)$$

where $T_{\max}^k$ denotes the maximum temperature and $T_{g(u,v)}^k$ represents the temperature at $g(u, v)$ in b_k . Higher temperatures at $g(u, v)$ result in lower costs, increasing the probability of placing a TSV at that location. $T_{g(u,v)}^k$ is calculated by the power blurring method (see Section II-B). The wirelength calculation method is also derived from HPWL.

In the TA stage, we use the same objective function as in (11) based on the wirelength and temperature functions. For brevity, we omit further elaboration on this topic. It is important to note that f_p represents a feasible TA solution at this stage.

B. Problem Formulation and Solution

Fig. 9(a) illustrates a four-chip tiers network structure, serving as an example to describe the TA problem. The figure showcases two nets: $\eta_1 = q_1, t_1$ and $\eta_2 = q_2, t_2$. Each net's signal path starts at a source pin, traverses the corresponding slots in each tier, and connects to the sink pin through TSVs, facilitating signal transmission. Specifically, net η_1 traverses from b_1 to b_3 , while net η_2 traverses from b_1 to b_4 . As a result, η_1 requires two TSVs, while η_2 requires three TSVs to establish vertical connections across the tiers of the entire net. Our objective is to optimize the TA positions for all nets, effectively minimizing wirelength and temperature on the chip. To explore improved TA strategies through local search, we treat each TSV tier as an agent and define a corresponding MDP for each agent.

To generate an adjacent TSV solution, we first define the action space of the TA problem. Fig. 9(b) illustrates the initial layout obtained after floorplanning, where the black dots represent the initial TSV positions. The search space for TSV positions is defined by $r = 1$, $r = 2$, and $r = 3$. Four perturbation methods were employed: 1) Swap a random pair of TSVs in $r = 1$; 2) swap a random pair of TSVs in $r = 2$; 3) swap a random pair of TSVs in $r = 3$; and 4) shift a randomly selected TSV to a new position in the range of $1 \leq r \leq 3$. The state space S represents a complete solution, including the TSV placement strategy of an agent (TSV tier). Applying any of the aforementioned perturbation methods to a TA solution (state) causes the agent to transition to a different state. Furthermore, we adopt the reward system expressed in (12) to maximize the total reward in this stage. We formulate the local search problem for TA.

Problem 2: Given the above MDP model, the RL agents are to find their best policies $\pi^* = \{\pi_1^*, \dots, \pi_{K-1}^*\}$ so that the wirelength and the temperature of the TA are minimized.

In this stage, we employ the ATT-MADDPGA algorithm to solve the TA problem. Unlike in the GD stage, each agent in this stage corresponds to a TSV tier.

VII. EXPERIMENTS RESULTS

A. Simulation Setup

The proposed framework is implemented in Python with the Pytorch library [42] and validated on the 2.5 GHz Intel Xeon CPU under the Linux workstation with 180 GB memory. We took GSRC benchmarks [43] and IBM-HB+ benchmarks [44] to test our algorithm, including three GSRC circuits (n100, n200, n300) and three IBM-HB+ circuits (ibm01, ibm03, ibm07). The chip's aspect ratio is set to 1:1, with a 15% whitespace and an elementary unit of $1 \mu\text{m}$. In all test cases, the TSV size is fixed at $3 \mu\text{m} \times 3 \mu\text{m}$. Temperature results are obtained using HotSpot [45], with an ambient temperature of 293 K.

TABLE I
COMPARISON WITH DIFFERENT TIER ASSIGNMENT METHODS

No. Of tiers	Test case	Power density-aware F-M in [13]						<i>k</i> -means						DCN					
		Tier Assignment		Thermal-aware Floorplanning				Tier Assignment		Thermal-aware Floorplanning				Tier Assignment		Thermal-aware Floorplanning			
		No. Of TSV	Time (s)	WL (μ m)	Temperature (C)			No. Of TSV	Time (s)	WL (μ m)	Temperature (C)			No. Of TSV	Time (s)	WL (μ m)	Temperature (C)		
					max	min	range				max	min	range				max	min	range
3	n100	700	0.91	211627	28.66	20.78	7.88	679	1.25	205014	28.82	21.03	7.79	694	1.40	196158	26.31	20.84	5.47
3	n200	1529	3.81	335328	29.58	20.74	8.84	1145	2.63	335462	27.78	20.74	7.04	1189	3.19	350260	28.04	20.81	7.23
3	n300	1403	5.71	442460	33.18	20.48	12.70	1340	4.28	447241	37.21	21.03	16.18	1391	4.88	429066	30.96	20.47	10.49
4	n100	1051	1.05	176042	31.12	20.59	10.53	1118	1.45	173327	32.53	20.42	12.11	1160	1.63	183286	29.08	20.61	8.47
4	n200	1903	5.08	322336	35.35	20.92	14.43	1384	3.19	316801	36.15	20.69	15.46	1439	3.43	342041	32.48	20.87	11.61
4	n300	2322	5.70	395578	37.41	20.87	16.54	1895	4.62	391383	36.72	20.74	15.98	2003	5.27	420420	33.55	20.81	12.74
	Nor.	1.13	1.12	0.98	1.08	1.00	1.27	0.96	0.88	0.97	1.10	1.00	1.33	1	1	1	1	1	1

The training process involves 200 episodes, with a maximum of 2000 training steps per episode. Multiple training instances are executed within each episode. The DCN's forward network consists of five layers (see Fig. 2), while the reconstruction network follows a mirrored structure. To balance the reconstruction error and clustering regularization, we set $\lambda = 0.1$. To prevent overfitting, we implement an early stopping mechanism that terminates the process if no improvement is observed within the last 50 steps, indicating policy convergence. The critics utilize 4-head attention networks as the default configuration, and the actors employ feed-forward networks with two hidden layers, each comprising 32 neurons. Recent experiences are stored in a replay memory of size 100 000, from which a batch of 128 experiences is sampled during training. The actor's learning rate is set to 0.001, the critic's learning rate to 0.01, the target network's learning rate to $\tau = 0.001$, and the discount factor to $\gamma = 0.95$. To ensure training stability, the target network remains fixed, and the trained policy network is synchronized with it every ten episodes. The network models were trained using the adaptive moment estimation (Adam) optimizer [46], with exponential decay rates for the moment estimates set to 0.9 and 0.999, respectively.

B. Dataset Generation

To assess the impact of the training dataset on our framework's performance, we generate random netlists as training samples. We create three datasets and train corresponding network models for each dataset. The small models are trained using netlists consisting of 100 modules with widths and heights ranging from 10 to 100, along with assigned power consumption values. Each module has three pins at its center, and we form 100 signals (nets) by randomly connecting three pins. For the medium model, we utilize netlists comprising 300 modules, each with six pins, and 300 nets. The remaining settings remain consistent with the small model case. In addition, we constructed a larger dataset containing 600 modules, ten pins, and 2000 nets. Despite the longer training time associated with the larger dataset, we chose it for training the network model due to its superior performance and the one-time training effort, as discussed in Section VII-E. Finally, we evaluate the model using both GSRC and IBM-HB+ benchmarks.

C. Ablation Study

In this section, we first examine the impact of power density-aware tier assignment on the temperature of 3-D ICs.

We employ the deep *k*-means clustering technique to achieve improved module clustering and temperature reduction. We compare our method with the power density-aware tier assignment algorithm based on FM proposed by Lin et al. [13], as it represents the state-of-the-art thermal-aware tier assignment approach. Furthermore, a comparison is made with the power density-aware tier assignment algorithm using the *k*-means clustering technique, which lacks a DNN for dimensionality reduction of input module features as employed in the deep *k*-means approach. Notably, the experiment follows our methodology, with the exception of the tier assignment method.

The experimental results, presented in Table I, display power distribution outcomes achieved through FM and *k*-means clustering in columns 3–8 and 9–14, respectively, while column 15–20 showcases our own results. Observing the table, it becomes evident that the FM-based approach achieves a 2% reduction in wirelength compared to ours. However, it registers a 13% higher TSV count and an 8% higher maximum temperature. Furthermore, the temperature range of the FM method is 1.27 times larger than that of our proposed approach. These results underline our method's capability to optimize power distribution and effectively mitigate temperature fluctuations. This is because the deep *k*-means clustering algorithm used in this study is an unsupervised ML method that outperforms the traditional FM algorithm utilized in the baseline. Deep *k*-means capture subtle and complex features, while FM relies on a predefined graph structure and iterative moves, limiting its adaptability and leading to suboptimal temperature reduction. In addition, although the wirelength and TSV count achieved by the *k*-means clustering method is 3% and 4% lower than the deep *k*-means clustering method, the maximum temperature obtained through *k*-means clustering is 10% higher. The experimental results demonstrate that deep *k*-means clustering with the DCN architecture can effectively reduce the dimensionality of input module features, leading to better clustering results compared to *k*-means clustering. As a result, cross-tier power distribution is improved, resulting in a notable temperature reduction in 3-D ICs. In addition, our method is highly efficient, with a maximum runtime of only 5.27 s (as shown in column 10).

Second, we assess the impact of the GCN and the attention mechanism in our approach on performance. Table II provides a summary of their contributions. We perform 100 complete runs of our algorithm for each benchmark and record the average statistical results. Columns 3–7 present the framework without GCN, where the input to the multiagent system lacks graph feature embeddings. Columns 8–12 display the

TABLE II
EVALUATION OF GCN AND ATTENTION MECHANISM PERFORMANCE

No. Of tiers	Test case	W/O. GCN						W/O. Attention Mechanism						Our Method					
		WL (μm)	Time (s)	Temperature (C)			WL (μm)	Time (s)	Temperature (C)			WL (μm)	Time (s)	Temperature (C)					
				max	min	range			max	min	range			max	min	range			
3	n100	221893	4.83	28.92	21.02	7.90	201814	4.74	27.48	20.82	6.66	196158	4.91	26.31	20.84	5.47			
3	n200	382538	8.13	30.31	20.84	9.47	361675	8.42	28.76	20.81	7.95	350260	8.54	28.04	20.81	7.23			
3	n300	473384	18.54	33.83	20.83	13.00	451562	19.65	32.42	20.28	12.14	429066	19.04	30.96	20.47	10.49			
4	n100	194432	6.16	31.86	20.79	11.07	191821	6.19	30.33	20.81	9.52	183286	6.31	29.08	20.61	8.47			
4	n200	380451	9.87	34.81	21.02	13.79	357658	12.05	33.57	21.09	12.48	342041	10.22	32.48	20.87	11.61			
4	n300	460704	35.31	36.97	20.94	16.03	433581	34.39	36.92	20.86	16.06	420420	38.16	33.55	20.81	12.74			
Nor.		1.10	0.95	1.09	1.01	1.27	1.04	0.98	1.05	1	1.16	1	1	1	1	1			

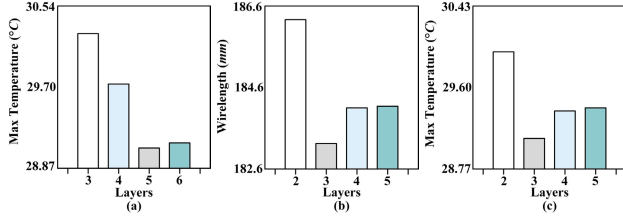


Fig. 10. Wirelength and temperature comparisons among different layer numbers of (a) DCN, (b) GCN, and (c) GCN (n100 benchmark) in a 3-D IC with four tiers.

results using MADDPG without the attention mechanism. Columns 13–17 illustrate the results obtained with our complete technique. From the table, we observe that while the network architecture without GCN takes 5% less time than our proposed method with GCN, the wirelength and maximum temperature are 10% and 9% higher, respectively. The results demonstrate that our proposed network architecture with GCN outperforms the framework without GCN, indicating that GCN extracts more efficient feature representations than the original state features. Consequently, the multiagent system can generalize to new, unseen netlists.

In addition, we note from the table that the method without an attention mechanism achieves shorter execution times. However, compared to our attention-based approach, it yields higher wirelength and maximum temperature values by 4% and 5%, respectively. The results indicate that our proposed attention-based algorithm surpasses the MADDPG algorithm without the attention mechanism, highlighting the effectiveness of our attention mechanism in modeling teammate strategies and facilitating effective cooperation among agents, thereby effectively reducing chip temperature and wirelength.

In the final ablation experiment, we investigate the impact of the number of layers in the DCN and GCN networks, as well as the hyperparameter K , on the performance of our framework. Fig. 10 illustrates the changes in wirelength and temperature on the n100 benchmark for different numbers of layers in the DCN and GCN. Importantly, when analyzing the influence of layer numbers in one network on performance, the layer numbers in the other network remain constant. The histogram in Fig. 10 demonstrates that as the depth increases, the wirelength and maximum temperature of the chip initially reach a saturation point and then gradually increase. This can be attributed to the observed degradation problem in neural networks [47]. Therefore, it is appropriate to set the layer numbers of the DCN and the GCN to 5 and 3. Fig. 11(a) and (b) present the changes in wirelength and temperature on

the n100 benchmark in a 3-D IC with four tiers in relation to the hyperparameter K , respectively. The histogram reveals a pattern where increasing K leads to a gradual saturation followed by a subsequent increase in wirelength and temperature. Therefore, setting $K = 4$, as depicted in Fig. 8, is justified. In addition, we conducted similar ablation experiments to determine the optimal network size and number of layers for both the policy network and the value network within the multiagent system.

D. Comparison With Baseline

In this section, we present a comparative analysis of our 3-D IC floorplanning methodology with the approach in [13], using GSRC and IBM-HB+ benchmarks. We specifically compare our approach with this baseline, as it is the only state-of-the-art three-stage 3-D IC floorplanner that can simultaneously place TSVs and mixed-typed modules. All programs were evaluated using identical benchmarks and power profile settings. We generated 100 test cases for each benchmark by randomly assigning the locations of the I/O bumps. Therefore, the results are the average values of 100 runs of each benchmark.

Table III provides a comprehensive comparison between the two methods using the GSRC benchmarks. Columns 3–8 present the results of the baseline method, while columns 9–14 display our results. Notably, column 14 presents the runtime of our method during each of the three stages. Our approach achieves remarkable improvements, including a 22% reduction in wirelength, a 13% reduction in the number of TSVs, a 24% decrease in maximum temperature, a 72% decrease in temperature range, and a 48% reduction in running time compared to the baseline. Moving on to Table IV, it presents a detailed comparison using the IBM-HB+ benchmarks. Our approach outperforms the baseline method by achieving a significant 52% reduction in maximum temperature, a 61% reduction in wirelength, a 44% reduction in the number of TSVs, and a remarkable 3.65 times faster running time.

To visually demonstrate the effectiveness of our methodology, Fig. 12 displays the floorplans and temperature maps of n100 with three tiers. The left and right sides of the figure depict the results of the baseline and our approach, respectively. Notably, our approach achieves more uniform temperature distributions and lower maximum temperatures compared to the baseline.

Our method achieves superior performance due to several key factors. First, we utilize a deep k -means clustering algorithm during the tier assignment stage, which allows for better cross-tier power distribution. Section VII-C highlights

TABLE III
COMPARISON OUR METHODOLOGY WITH BASELINE IN GSRC BENCHMARKS

No. Of tiers	Test case	Methodology in [13]						Our Methodology					
		No. Of TSV	WL (μm)	Time (s)	Temperature (C)			No. Of TSV	WL (μm)	Temperature (C)			Time (s)
					max	min	range			max	min	range	
3	n100	700	236969	7.35	34.41	21.17	13.24	694	196158	26.31	20.84	5.47	1.40+2.41+1.10=4.91
3	n200	1529	422630	13.40	32.81	21.17	11.64	1189	350260	28.04	20.81	7.23	3.19+4.11+1.24=8.54
3	n300	1403	567994	28.28	37.75	21.10	16.65	1391	429066	30.96	20.47	10.49	4.88+10.09+4.07=19.04
4	n100	1051	211889	8.01	36.64	20.96	15.68	1160	183286	29.08	20.61	8.47	1.63+3.06+1.62=6.31
4	n200	1903	405571	15.11	40.92	21.24	19.68	1439	342041	32.48	20.87	11.61	3.43+4.98+1.81=10.22
4	n300	2322	498850	56.49	41.03	21.40	19.63	2003	420420	33.55	20.81	12.74	5.27+20.62+12.27=38.16
Nor.		1.13	1.22	1.48	1.24	1.02	1.72	1	1	1	1	1	1

TABLE IV
COMPARISON OUR METHODOLOGY WITH BASELINE IN IBM-HB+ BENCHMARKS

No. Of tiers	Test case	Methodology in [13]						Our Methodology					
		No. Of TSV	WL ($10^3\mu\text{m}$)	Time (s)	Temperature (C)			No. Of TSV	WL ($10^3\mu\text{m}$)	Temperature (C)			Time (s)
					max	min	range			max	min	range	
3	ibm01	2603	5.67	248.63	54.08	21.83	32.25	3993	3.65	35.84	21.95	13.89	19.95+35.46+18.48=73.89
3	ibm03	4880	14.14	368.39	64.16	22.09	42.07	6146	8.83	42.32	21.22	21.10	28.65+46.44+23.72=98.81
3	ibm07	18554	25.26	649.91	74.20	22.93	51.27	8877	14.41	46.39	21.98	24.41	46.42+84.54+34.82=165.78
4	ibm01	3254	5.11	275.33	57.94	21.45	36.49	5318	3.04	40.35	21.31	19.04	22.35+39.10+18.36=79.81
4	ibm03	5903	11.32	411.27	74.81	22.02	52.79	6684	8.47	49.27	21.07	28.20	39.06+56.14+26.86=122.06
4	ibm07	25157	20.84	751.06	84.16	22.17	61.99	10912	12.73	55.62	21.15	34.47	48.13+110.31+42.12=200.56
Nor.		1.44	1.61	3.65	1.52	1.03	1.96	1	1	1	1	1	1

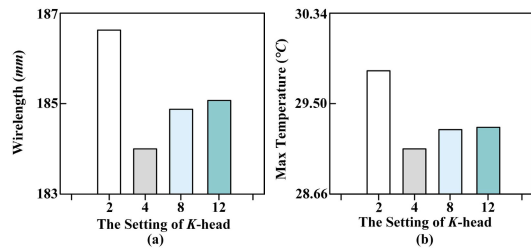


Fig. 11. Changes in (a) wirelength and (b) temperature on the n100 benchmark in a 3-D IC with four tiers with respect to the hyperparameter K .

the significant impact of tier assignment on 3-D IC temperature. Second, in the GD and TA stages, our approach combines GCN with the MADRL algorithm, forming an end-to-end learning framework. Compared to traditional analytical methods within a multilevel framework, our approach significantly reduces algorithmic complexity. By effectively capturing complex patterns and dependencies in chip layouts, GCN empowers agents to make informed decisions. Moreover, our incorporation of an attention mechanism in the centralized critic facilitates efficient cooperation among agents, resulting in better floorplanning outcomes with reduced wirelength and temperature in 3-D ICs. Furthermore, unlike their method, which employs a sequential TA approach within the MCMF-based algorithm where assigned TSVs become fixed and irreplaceable, our approach avoids reliance on the net order, mitigating potential detrimental outcomes.

E. Effect of Training Dataset on Performance

In this section, we investigate the influence of the training dataset on the learning capability of our method. We train the framework using three distinct datasets, as described in Section VII-B, and evaluate the models using representative examples such as n100, n200, and n300 circuits. Fig. 13 displays two bar graphs that illustrate the impact of the training datasets on wirelength and temperature. Notably, our method achieves superior results with the increasing size of the training data. This finding suggests that utilizing larger and more diverse chip netlists for training mitigates overfitting tendencies and improves generalization to unseen designs. Moreover, the training time for the three datasets is approximately 6, 17,

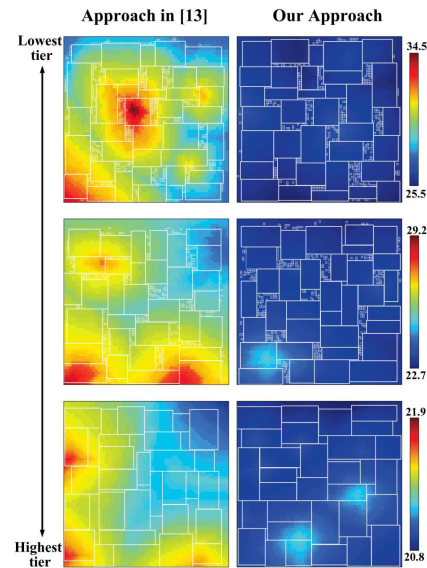


Fig. 12. Resulting floorplans and the associated temperature maps of n100 with three tiers, where the left and right columns, respectively, show the results of [13] and our methodology.

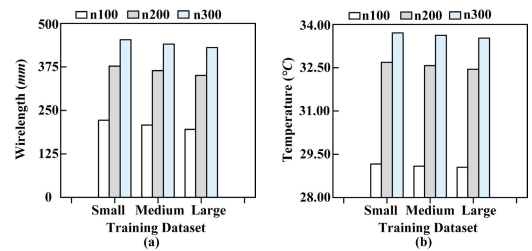


Fig. 13. Impact of training dataset on (a) wirelength and (b) temperature.

and 29 h, respectively. Despite the longer training time for the larger dataset, we selected it for both GSRC and IBM-HB+ benchmarks due to its enhanced performance and the one-time training effort required to calibrate our model.

VIII. CONCLUSION

This article proposes an end-to-end, learning-based thermal-aware fixed-outline floorplanning approach for 3-D ICs. We begin by employing a power density-aware deep k -means

clustering algorithm to allocate modules to suitable tiers. Then, we combine GCN with the ATT-MADDPG algorithm to optimize the positions for modules and TSVs while minimizing wirelength and temperature. Finally, we utilize the ATT-MADDPG algorithm again to refine the TSV positions, further reducing the wirelength and temperature of the chip. Experimental results demonstrated that compared with the state-of-the-art heuristic-based method, our approach can obtain better results.

Our work has overlooked power/ground TSVs (P/G TSVs) and TTSVs. In future work, we will extend our methodology to encompass a comprehensive thermal analysis that takes into account signal TSVs, P/G TSVs, and TTSVs simultaneously, aiming to enhance the efficient dissipation of heat in 3-D ICs.

REFERENCES

- [1] J. Cong, J. Wei, and Y. Zhang, "A thermal-driven floorplanning algorithm for 3D ICs," in *Proc. ICCAD*, Nov. 2004, pp. 306–313.
- [2] P. Zhou et al., "3D-STAF: Scalable temperature and leakage aware floorplanning for three-dimensional integrated circuits," in *Proc. ICCAD*, Nov. 2007, pp. 590–597.
- [3] E. Wong and S. K. Lim, "3D floorplanning with thermal vias," in *Proc. DATE*, Mar. 2006, pp. 6–10.
- [4] L. Xiao, S. Sinha, J. Xu, and E. F. Y. Young, "Fixed-outline thermal-aware 3D floorplanning," in *Proc. ASP-DAC*, Jan. 2010, pp. 561–567.
- [5] Q. Xu and S. Chen, "Fast thermal analysis for fixed-outline 3D floorplanning," *Integration*, vol. 59, pp. 157–167, Sep. 2017.
- [6] T. Ni et al., "Temperature-aware floorplanning for fixed-outline 3D ICs," *IEEE Access*, vol. 7, pp. 139787–139794, 2019.
- [7] J. Knechtel, E. F. Y. Young, and J. Lienig, "Planning massive interconnects in 3-D chips," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 11, pp. 1808–1821, Nov. 2015.
- [8] X. Hong et al., "Corner block list: An effective and efficient topological representation of non-slicing floorplan," in *Proc. ICCAD*, Nov. 2000, pp. 8–12.
- [9] A. Ziabari, J.-H. Park, E. K. Ardestani, J. Renau, S.-M. Kang, and A. Shakouri, "Power blurring: Fast static and transient thermal analysis method for packaged integrated circuits and power devices," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 22, no. 11, pp. 2366–2379, Nov. 2014.
- [10] Y. Chen, E. Kursun, D. Motschman, C. Johnson, and Y. Xie, "Through silicon via aware design planning for thermally efficient 3-D integrated circuits," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 9, pp. 1335–1346, Sep. 2013.
- [11] D. Oh, C. C. P. Chen, and Y. H. Hu, "Efficient thermal simulation for 3-D IC with thermal through-silicon vias," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 31, no. 11, pp. 1767–1771, Nov. 2012.
- [12] Z. Ren, A. Alqahtani, N. Bagherzadeh, and J. Lee, "Thermal TSV optimization and hierarchical floorplanning for 3-D integrated circuits," *IEEE Trans. Compon., Packag., Manuf. Technol.*, vol. 10, no. 4, pp. 599–610, Apr. 2020.
- [13] J.-M. Lin, W.-Y. Chang, H.-Y. Hsieh, Y.-T. Shyu, Y.-J. Chang, and J.-M. Lu, "Thermal-aware floorplanning and TSV-planning for mixed-type modules in a fixed-outline 3-D IC," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 9, pp. 1652–1664, Sep. 2021.
- [14] D. Vashisht et al., "Placement in integrated circuits using cyclic reinforcement learning and simulated annealing," 2020, *arXiv:2011.07577*.
- [15] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [16] Y.-C. Lu, S. Pentapati, L. Zhu, G. Murali, K. Samadi, and S. K. Lim, "A machine learning-powered tier partitioning methodology for monolithic 3-D ICs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 11, pp. 4575–4586, Nov. 2022.
- [17] A. Mirhoseini et al., "A graph placement methodology for fast chip design," *Nature*, vol. 594, no. 7862, pp. 207–212, Jun. 2021.
- [18] Q. Xu et al., "GoodFloorplan: Graph convolutional network and reinforcement learning-based floorplanning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 10, pp. 3492–3502, Oct. 2022.
- [19] Z. He et al., "Learn to floorplan through acquisition of effective local search heuristics," in *Proc. ICCD*, Oct. 2020, pp. 324–331.
- [20] K. Zhu, M. Liu, H. Chen, Z. Zhao, and D. Z. Pan, "Exploring logic optimizations with reinforcement learning and graph convolutional network," in *Proc. ACM/IEEE 2nd Workshop Mach. Learn. CAD (MLCAD)*, Nov. 2020, pp. 145–150.
- [21] A. Goldie and A. Mirhoseini, "Placement optimization with deep reinforcement learning," in *Proc. ACM Int. Symp. Phys. Design*, Mar. 2020, pp. 3–7.
- [22] H. Liao, W. Zhang, X. Dong, B. Poczoz, K. Shimada, and L. B. Kara, "A deep reinforcement learning approach for global routing," *J. Mech. Des.*, vol. 142, no. 6, pp. 1–16, Jun. 2020.
- [23] H. Liao et al., "Attention routing: Track-assignment detailed routing using attention-based reinforcement learning," 2020, *arXiv:2004.09473*.
- [24] H. Mao, Z. Zhang, Z. Xiao, and Z. Gong, "Modelling the dynamic joint policy of teammates with attention multi-agent DDPG," 2018, *arXiv:1811.07029*.
- [25] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2000, pp. 1057–1063.
- [26] I. Grondman, L. Busoniu, G. A. D. Lopes, and R. Babuska, "A survey of actor-critic reinforcement learning: Standard and natural policy gradients," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 42, no. 6, pp. 1291–1307, Nov. 2012.
- [27] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. PMLR*, 2014, pp. 387–395.
- [28] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [29] B. Yang, X. Fu, N. D. Sidiropoulos, and M. Hong, "Towards K-means-friendly spaces: Simultaneous deep learning and clustering," in *Proc. 34th Int. Conf. Mach. Learn.*, 2016, pp. 3861–3870.
- [30] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. ICML*, 2010, pp. 807–814.
- [31] Y. Bengio, P. L. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," in *Proc. NIPS*, Dec. 2007, pp. 1–8.
- [32] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986.
- [33] D. Sculley, "Web-scale k-means clustering," in *Proc. WWW*, Apr. 2010, pp. 1177–1178.
- [34] X. He, S. Dong, Y. Ma, and X. Hong, "Simultaneous buffer and interlayer via planning for 3D floorplanning," in *Proc. 10th Int. Symp. Quality Electron. Design*, Mar. 2009, pp. 740–745.
- [35] P. H. Shiu, R. Ravichandran, S. Easwar, and S. K. Lim, "Multi-layer floorplanning for reliable system-on-package," in *Proc. ISCAS*, Vancouver, BC, Canada, 2004, p. V.
- [36] H. Murata, K. Fujiyoshi, S. Nakatake, and Y. Kajitani, "VLSI module placement based on rectangle-packing by the sequence-pair," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 15, no. 12, pp. 1518–1524, Dec. 1996.
- [37] X. Tang, R. Tian, and D. F. Wong, "Fast evaluation of sequence pair in block placement by longest common subsequence computation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 20, no. 12, pp. 1406–1413, Dec. 2001.
- [38] S. K. Lim, *Practical Problems in VLSI Physical Design Automation*. Dordrecht, The Netherlands: Springer, 2008.
- [39] J. Zhou et al., "Graph neural networks: A review of methods and applications," 2018, *arXiv:1812.08434*.
- [40] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *PMLR*, Jul. 2018, pp. 1587–1596.
- [41] X. Liu, G. Yeap, J. Tao, and X. Zeng, "Integrated algorithm for 3-D IC through-silicon via assignment," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 22, no. 3, pp. 655–666, Mar. 2014.
- [42] A. Paszke et al., "Automatic differentiation in PyTorch," in *Proc. NIPS Workshop*, 2017, pp. 1–4.
- [43] W. Dai, L. Wu, and S. Zhang. (2000). *GSRC Benchmarks*. [Online]. Available: <http://vlsicad.eecs.umich.edu/BK/GSRCbench>
- [44] A. N. Ng, R. Aggarwal, V. Ramachandran, and I. Markov. (2006). *IBM-HB+ Benchmarks*. [Online]. Available: <http://vlsicad.eecs.umich.edu/BK/ISPD06bench/>
- [45] R. Zhang, M. R. Stan, and K. Skadron. (2015). *HotSpot 6.0 Temperature Modeling Tool*. [Online]. Available: <http://lava.cs.virginia.edu/HotSpot/>
- [46] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [47] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, Jun. 2016, pp. 770–778.